

Functions

Functions are the most important aspect of an application.

A function can be defined as the organized block of reusable code, which can be called whenever required.

Python allows us to divide a large program into the basic building blocks known as a function. The function contains the set of programming statements enclosed by {}. A function can be called multiple times to provide reusability and modularity to the Python program. Functions are the most important aspect of an application. A function can be defined as the organized block of reusable code, which can be called whenever required.

The function contains the set of programming statements enclosed by {}. A function can be called multiple times to provide reusability and modularity to the Python program. The Function helps to programmer to break the program into the smaller part. It organizes the code very effectively and avoids the repetition of the code. As the program grows, function makes the program more organized.

Python provide us various inbuilt functions like range() or print(). Although, the user can create its functions, which can be called user-defined functions.

There are mainly two types of functions.

User-defined functions: - The user-defined functions are those defined by the user to perform the specific task.

Built-in functions: - The built-in functions are those functions that are pre-defined in Python.

Advantage of Functions in Python

There are the following advantages of Python functions.

1. Using functions, we can avoid rewriting the same logic/code again and again in a program. We can call Python functions multiple times in a program and anywhere in a program.
2. We can track a large Python program easily when it is divided into multiple functions.
3. Reusability is the main achievement of Python functions.

Creating a Function

Python provides the `def` keyword to define the function.

The syntax of the define function is given below.

Syntax:

```
def my_function(parameters):  
    function_block  
    [return expression]
```

Let's understand the syntax of functions definition. The def keyword, along with the function name is used to define the function. The identifier rule must follow the function name. A function accepts the parameter (argument), and they can be optional. The function block is started with the colon (:), and block statements must be at the same indentation.

The return statement is used to return the value. A function can have only one return

Function Calling

In Python, after the function is created, we can call it from another function. A function must be defined before the function call; otherwise, the Python interpreter gives an error. To call the function, use the function name followed by the parentheses.

```
#function definition
def hello():
    print("hello world...bye")
```

```
# function calling
hello()
```

output:
hello world...bye

The return statement

The return statement is used at the end of the function and returns the result of the function. It terminates the function execution and transfers the result where the function is called. The return statement cannot be used outside of the function.

Syntax:

return [expression_list]

It can contain the expression which gets evaluated and value is returned to the caller function. If the return statement has no expression or does not exist itself in the function then it returns the object.

Defining function

```
def sum_func():#dont use the name sum, sum is a built function
```

```
    #redefining sum alter its functionality
```

```
    myvar1 = 10
```

```
    myvar2 = 20
```

```
    myvar = myvar1+myvar2
```

```
    return myvar
```

calling sum_func() function in print statement

```
print("The sum is:",sum_func())
```

output:

The sum is: 30

The return statement with multiple values

In Python, functions can return multiple values by returning them as a tuple.

This is a common and powerful feature.

```
def min_max(numbers):
```

```
    min_value=min(numbers)
```

```
    max_value=max(numbers)
```

```
    return(min_value,max_value) # you can return multiple values
```

```
nums = [3, 7, 2, 9, 4]
```

```
low, high = min_max(nums) # value assigned in same order as returned
```

```
print("Min:", low) # Output: Min: 2
```

```
print("Max:", high) # Output: Max: 9
```

Creating function without return statement

In Python, you **can create functions without a return statement**. These are often used when the function's purpose is to perform an **action**, like printing, modifying data, or saving a file, rather than calculating and returning a value.

```
# Defining function
def sum_func():#dont use the name sum, sum is a built function
    #redefining sum alter its functionality
    myvar1 = 10
    myvar2 = 20
    myvar = myvar1+myvar2
```

```
# calling sum_func() function in print statement
print("The sum is:",sum_func())
```

output:
None

In the above code, we have defined the same function without the return statement as we can see that the sum() function returned the None object to the caller function. In the definition nothing is returned, in this case usually value is print inside the function itself

Print inside the function

```
# Defining function
def sum_func():#dont use the name sum, sum is a built
function
    #redefining sum alter its functionality
    myvar1 = 10
```

```

myvar2 = 20
myvar = myvar1+myvar2
print("The sum is:",myvar)

# calling sum_func() function in print statement
sum_func()

```

Arguments in function

The arguments are types of information which can be passed into the function.

The arguments are specified in the parentheses. We can pass any number of arguments, but they must be separate them with a comma.

```

# defining the function
def sum_func(myvar1, myvar2): #arguments received are
    assigned to myvar1 and myvar2 respectively
    return(myvar1 + myvar2)

# taking values from the user
var1 = int(input("Enter var1:"))
var2 = int(input("Enter var2:"))

# printing the sum of a and b
print("Sum = ", sum_func(var1, var2))#var1 and var2 are
arguments that are passed to function

```

output:
Sum = 5

Passing mutable Object (List)

Mutable objects can be **changed in-place**. So if you pass a list to a function and modify it inside the function, the original list **outside** the function is also affected.

```
def change_list(mylist):
    mylist.append(20)
    mylist.append(30)
    print("list inside function = ", mylist)
```

```
# defining the list
mylist1 = [10, 30, 40, 50]
# calling the function
change_list(mylist1)
```

```
print("list outside function = ", mylist1)
```

output:

```
list inside function = [10, 30, 40, 50, 20, 30]
list outside function = [10, 30, 40, 50, 20, 30]
```

Passing Immutable Object (String)

Strings in Python are **immutable**, which means they **cannot be changed in place**. When you pass a string to a function, any modification to it **creates a new string**, and the original remains **unchanged** outside the function

```
# defining the function
def change_string(mystr):
    mystr = mystr + " Hows you "
    print("printing the string inside function :", mystr)
```

```
mystr1 = "Hi I am there"
# calling the function
change_string(mystr1)
print("printing the string outside function :", mystr1)
```

output:

```
printing the string inside function : Hi I am there Hows you
printing the string outside function : Hi I am there
```

Types of arguments

There may be several types of arguments which can be passed at the time of function call.

1. Required arguments
2. Keyword arguments
3. Default arguments
4. Variable-length arguments

We can provide the arguments at the time of the function call. As far as the required arguments are concerned, these are the arguments which are required to be passed at the time of function calling with the exact match of their positions in the function call and function definition. If either of the arguments is not provided in the function call, or the position of the arguments is changed, the Python interpreter will show the error.

#1. Required arguments

```
def func(name):  
    message= "Hi "+name  
    return message
```

```
nm = input("Enter name:")  
print(func(nm))
```

output:

Enter name:Alice

Hi Alice

returns the simple interest accordingly

#the function simple_interest accepts three arguments and returns the simple interest accordingly

```
def simple_interest(p,t,r):
    return (p*t*r)/100

p1 = float(input("Enter the principle amount:"))
r1 = float(input("Enter the rate of interest:"))
t1 = float(input("Enter the time in years:"))

print("Simple Interest: ",simple_interest(p1,t1,r1))
```

output:
Enter the principle amount:1000
Enter the rate of interest:5
Enter the time in years:5
Simple Interest: 250

#the function calculate returns the sum of two arguments
myvar1 and myvar2

```
def calculate(myvar1,myvar2):
    return myvar1+myvar2
print(calculate(3,5))
print(calculate(10)) # error
```

output:
8

```
-----
-----
TypeError                                Traceback (most
recent call last)
Cell In[14], Line 6
      4     return myvar1+myvar2
      5     print(calculate(3,5))
----> 6     print(calculate(10)) # error
```

TypeError: calculate() missing 1 required positional
argument: 'myvar2'

Default Arguments

Python allows us to initialize the arguments at the function definition. If the value of any of the arguments is not provided at the time of function call, then that argument can be initialized with the value given in the definition even if the argument is not specified at the function call.

```
def printme(name,age=22):  
    print("My name is",name,"and age is",age)  
  
printme(name = "john") #the variable age is not passed
```

output:

My name is john and age is 22

Variable-length Arguments (*args)

In large projects, sometimes we may not know the number of arguments to be passed in advance. In such cases, Python provides us the flexibility to offer the comma-separated values which are internally treated as tuples at the function call. By using the variable-length arguments, we can pass any number of arguments.

However, at the function definition, we define the variable-length argument using the *args (star) as *<variable - name >.

```
def printme(*names):  
    for name in names:  
        print(name)  
  
printme("john","David","smith","nick")
```

```
printme(10,20)
```

output:

```
john  
David  
smith  
nick  
10  
20
```

*In the above code, we passed *names as variable-length argument. We called the function and passed values which are treated as tuple internally. The tuple is an iterable sequence the same as the list. To print the given values, we iterated *arg names using for loop.*

Keyword arguments(kwargs)**

Python allows us to call the function with the keyword arguments. This kind of function call will enable us to pass the arguments in the random order.

The name of the arguments is treated as the keywords and matched in the function calling and definition. If the same match is found, the values of the arguments are copied in the function definition.

#function func is called with the name and message as the keyword arguments

```
def func(name,message):  
    print("printing the message with",name,"and ",message)
```

#name and message is copied with the values John and hello respectively

```
func(name = "John",message="hello")
```

output:

printing the message with John and hello

Providing the values in different order at the calling

#The function `simple_interest(p, t, r)` is called with the keyword arguments
arguments the order of arguments doesn't matter in this case

```
def simple_interest(p,t,r):  
    return (p*t*r)/100  
  
print("Simple Interest: ",simple_interest(t=5,r=5,p=1000))
```

output:

Simple Interest: 250.0

If we provide the different name of arguments at the time of function call, an error will be thrown.

#The function `simple_interest(p, t, r)` is called with the keyword arguments.

```
def simple_interest(p,t,r):  
    return (p*t*r)/100  
print("Amt:",simple_interest(t=5,p=1000,r=5))
```

output:

Amt: 250.0

```
print("Simple Interest:  
",simple_interest(time=5,rate=5,principle=1000))
```

output:

TypeError
recent call last)
Cell In[22], line 1

Traceback (most

```
----> 1 print("Simple Interest:  
",simple_interest(time=5,rate=5,principle=1000))
```

TypeError: simple_interest() got an unexpected keyword argument 'time'

The Python allows us to provide the mix of the required arguments and keyword arguments at the time of function call. However, the required argument must not be given after the keyword argument, i.e., once the keyword argument is encountered in the function call, the following arguments must also be the keyword arguments.

```
def func(name1,message,name2):  
    print("printing the message with",name1,",",message,",and",name2)  
  
#the first argument is not the keyword argument  
func("John",message="hello",name2="David")
```

output:

printing the message with John , hello ,and David

Python provides the facility to pass the multiple keyword arguments which can be represented as ****kwargs**. It is similar as the ***args** but it stores the argument in the dictionary format. This type of arguments is useful when we do not know the number of arguments in advance. Many arguments using Keyword argument

```
def food(**kwargs):  
    print(kwargs)  
food(a="Apple")  
food(fruits="Orange", Vagitables="Carrot")
```

```
output:
{'a': 'Apple'}
{'fruits': 'Orange', 'Vegetables': 'Carrot'}
```

Scope of variables

The scopes of the variables depend upon the location where the variable is being declared. The variable declared in one part of the program may not be accessible to the other parts.

In python, the variables are defined with the two types of scopes.

1. **Global variables**
2. **Local variables**

The variable defined outside any function is known to have a global scope, whereas the variable defined inside a function is known to have a local scope.

Local Variable

- Defined inside a function.
- Only accessible within that function.
- Created when the function starts and destroyed when it ends.

```
def my_function():
    x = 10 # local variable
    print(x)

my_function()
```

```
print(x) # This would raise an error: NameError: name 'x'
is not defined
```

output:
10

NameError Traceback (most recent call last)

Cell In[2], Line 6

```
3     print(x)
5 my_function()
```

----> 6 print(x) # This would raise an error: NameError:
name 'x' is not defined

NameError: name 'x' is not defined

Global Variable

- Defined outside any function.
- Accessible from any function in the same module (file).
- Can be read inside functions, but to modify it inside a function, you must use the global keyword.

```
x = 5 # global variable
```

```
def my_function():
    global x
    x = 10 # modifies the global variable
    print(x)
```

```
my_function()
print(x)
```

output:
10
10